ScholarScout: An academic scholarship and admission planning AI agent

Haris Ali
sali.msds25seecs@seecs.edu.pk

Muhammad Uzair Siddiqui
muzair.msds25seecs@seecs.edu.pk

1 Proposed System

1.1 Data

The system covers three data sources:

- **Erasmus Mundus programmes**: Joint master programs funded by the European Union, scraped from the EACEA catalog. These are competitive, fully funded or partially funded international programmes offered by consortia of European universities.
- **German university programmes via DAAD**: International programmes at German universities listed on the DAAD (German Academic Exchange Service) database. Many are free for EU students and low-cost for non-EU students.
- **Professor research positions**: Manually curated open positions offered by experienced professors for PhD students, postdocs, and research assistants at European universities.

From all three sources, we collected programs over a variety of fields. This data was scrapped using selenium + beautifulsoup based scrappers. The web page data was further parsed and stored into structured database. Total 2,282 program records with fields for deadline, tuition fees (EU and non-EU), funding type, degree level, scholarship availability, and field category were stored in a PgSQL table. The semantic index covers program descriptions, including admission requirements, curriculum overviews, language requirements, and program-specific details.

For searching fellowship opportunities we built a second semantic index holds 100 professor opportunity records. Total we cover 31 field categories (Computer Science, Data Science, Environmental Science, Artificial Intelligence, Renewable Energy, etc.), defined manually based on the programe distribution in the scraped data.

1.2 System Architecture

ScholarScout is a multi-node state machine built with LangGraph. Every user message enters the pipeline and moves through a sequence of processing nodes before a response is generated. The main stages are:

1. **Profile extraction**: To allow better and more precise search we first collect user profile. For now this profile includes their degree and field of interest. For each input query the program first read and captures profile signals and stores the information in the conversation state without interrupting the flow.
2. **Query restoration**: Sometimes user is prompted to enter profile info before their input query could be processed. So in order to avoid asking them again their query is kept stored in state so once necessary info is obtainer we can then use the original question by recovering the original intent from stored state so it can be processed.
3. **Router**: For each query their are different retreival paths. For example a search for finances or deadlines cutoff doesn't require semantic search because in this scenario your postgresql search is much more precise and accurate. Similarly for your semantic based questions like *'Which courses focus more on AI in healthcare'* we do not need to hit the pgsql table. As a result we have divided these into different paths. For each user query we extract their embeddings and determines which particular path is suitable by comparing it with pre-computed anchors for each path and then run our retrieval system based on that path. This routing algorithm is intelligent and manages to carefully route the best path for a user query saving both time and resources.

4. **Retrieval node(s)**: Our retrieval system is based on two resources. One is our database built on PostgreSQL. It helps us filter out necessary field on interest, match deadlines and budget requirements. For semantic search we are using MILVUS. It is fast and production grade vector database. Here we use Flat indexing allowing us efficient retrieval of semantic field related information.

5. **Optional web search**: To make it a little more flexible we have added a web searching API from Tavily. The purpose here is to allow wide searches from web as well if something is not available right now in our database. But this option is entirely for user preference and can be configured from user side if they want to use it.

6. **Response generation**: For response generation and other llm based decision making we are using two llms. One is `gpt-4o` as our main backbone for generating final response grounded in the retrieved context. For small tasks of internal decision making based on context and for SQL query generation we use a smaller and faster llm `gpt-4o-mini`. This trade off is to manage both accuracy and response time of our application.

1.3 Faithful Response Generation

A common failure mode for a RAG is the model adding information that are not in the retrieved data. This is dangerous for our application because in that case misguiding information can have negative consequences for users. It will waste more time and will not be a quality user experience. In order to avoid this we have designed the response node by adding certain control checks:

- Deadlines are pre-labelled at indexing time as *upcoming* or *passed* based on the current date, so the LLM never computes relative dates itself.
- The response prompt instructs the model to only state what appears in the provided context and to explicitly acknowledge when data is unavailable rather than guessing.
- A structured output format (programme name, university, field, deadline, tuition, URL) constrains what the model can say, reducing the surface area for hallucination.

2 Comparison with Baselines

2.1 Evaluation Setup

We evaluated our system with 50 hand-labelled test queries distributed across five route categories: 14 structured filter queries (`query_db`), 12 semantic queries (`vector_search`), 10 ambiguous queries (`hybrid`), 8 general conversation queries (`respond`), and 6 research position queries (`professor_search`).

Five metrics were measured:

- **Routing accuracy**: Correct retrieval path selected by the router.
- **DB hit rate**: SQL query returned at least one result (for DB-routed queries).
- **Vector hit rate**: Semantic search returned at least one result (for vector-routed queries).
- **Judge score (0–3)**: Response quality evaluated by a GPT-4o judge on relevance, completeness, and accuracy.
- **Faithfulness rate**: Percentage of factual claims in the response traceable back to retrieved data.

Three baselines were defined for comparison:

- **B1 — Keyword Router**: Routes purely on keyword presence. Queries with financial terms ("tuition," "deadline," "funded," "scholarship")
- **B2 — DB-Only**: Always queries PostgreSQL, regardless of query type.
- **B3 — Vector-Only**: Always queries Milvus, regardless of query type.

We compare judge and faithfulness scores on these.

2.2 Quantitative Results

IN order to decide better retrieval strategy we tried to compare performance on different strategies. **TF-IDF** and **BM25** are both sparse embedding methods used to assign weights to keywords which are then used for search. **Hybrid Weighted** combines dense and BM25 scores using a fixed linear weight. **Hybrid RRF (Reciprocal Rank Fusion)** merges ranked lists from dense and sparse retrievers by reciprocal rank with no score calibration.

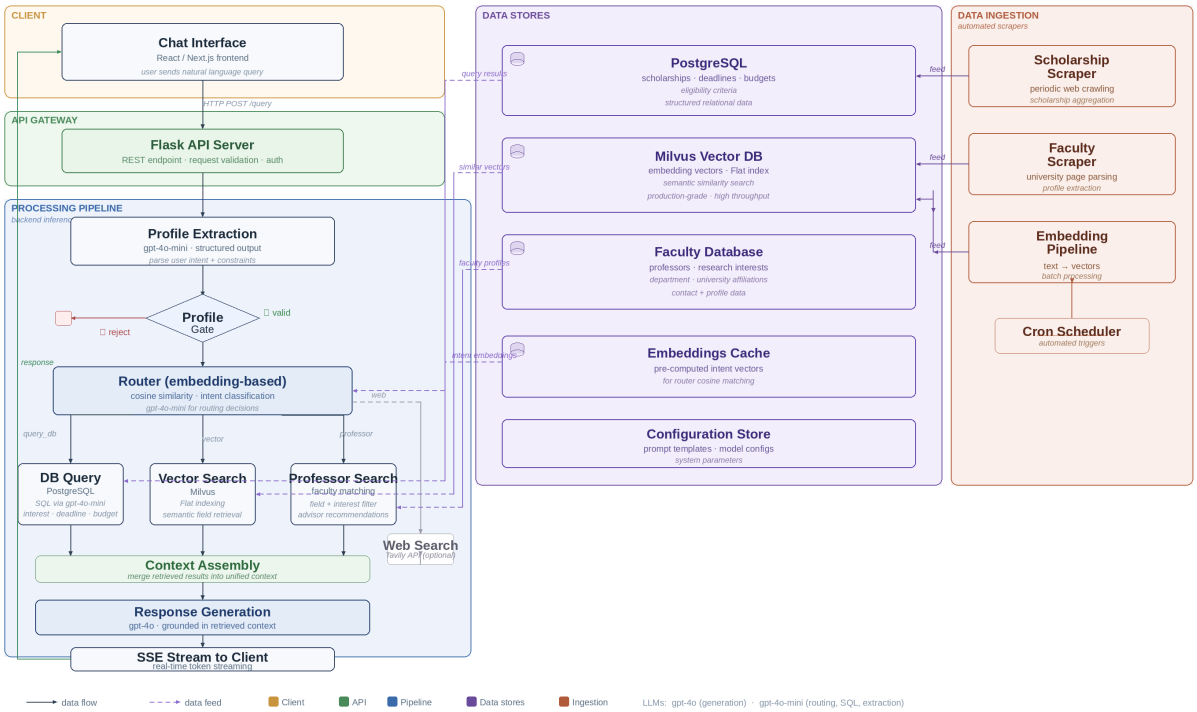

Figure 1: ScholarScout system architecture.

Table 1: Retrieval comparison.

	P@5	MRR	NDCG@5	ms
TF-IDF	0.80	1.00	0.972	9.7
BM25	0.84	1.00	0.975	1.1
Hybrid Wt.	0.92	1.00	0.978	8.6
Hybrid RRF	0.92	1.00	0.921	8.8
Dense (ours)	**0.94**	0.90	0.933	33.9

Table 2: Pipeline evaluation (50 queries).

	Judge	Faith.	Hit
DB-Only	1.65/3	91%	87.5%
Vec-Only	2.02/3	86%	95.5%
Ours	**2.48/3**	89%	–

Dense (ours) where queries and documents are encoded using a sentence transformer and retrieved via FAISS flat indexing on Milvus. We go with dense retrieval for better precision. Table 1 compares five retrieval strategies on the scholarship corpus.

ScholarScout's judge score of 2.48/3 is higher than both single-source strategies because it retrieves the right type of data for each query.

When a student asks "list tuition-free programmes in environmental science with upcoming deadlines," the database returns exact matches based on the filters with verified fee and deadline data. When they ask "what admission requirements do sustainability programmes typically have?" the semantic search returns relevant descriptions that the database cannot provide. Routing each query to the right source is the difference between a useful and an unhelpful answer.

The DB-only estimate of 1.65/3 reflects two things: structured filter queries score well (no structured data is missing), but semantic and hybrid queries produce either empty results or responses that lack any programme-specific content. The vector-only estimate of 2.02/3 is higher because descriptive text can partially satisfy structured queries (e.g., a description might mention a programme's tuition policy), but it cannot match the precision of actual fee data in the database.

Faithfulness is slightly higher for DB-only than for ScholarScout. This happens because structured database records are precise and leave little room for the model to add extra details. Semantic chunks are longer and less precise, which creates more opportunity for soft hallucination (adding general true summarized statements that are not in the specific retrieved chunk).

2.3 Qualitative Observations

The main role of hybrid retrieval is on boundary queries. A query like "show me fully funded AI programmes and explain what they cover" requires the use of both a filter component (fully funded) and a descriptive component (what they cover). The keyword router sends this

to the database which returns fully funded AI-category programmes but cannot explain their content. ScholarScout detects DB and semantic similarity scores are close and runs both, combining the structured list with semantic descriptions. The responses on these queries were noticeably more complete.

Profile enrichment has a visible effect on retrieval precision. With no profile context, "find programmes in my area" retrieves broadly across all fields. With a basic profile information appended ("degree level: Masters, field: Computer Science, specialization: Natural Language Processing") the same query returns a much tighter set of relevant results. This was confirmed by manually comparing retrieval outputs on 10 representative queries.

Not all failure cases hurt the user equally. Some routing errors end up not mattering much. A query like "what programmes are available in environmental science?" was misrouted to `hybrid` instead of `query_db` but since hybrid runs both retrievals the user still got a comprehensive answer. The routing error was invisible from the user perspective. Other errors are more costly — routing a semantic query to database-only means the user gets a flat list with no descriptive content.

3 Trade-offs and Constraints

3.1 Accuracy vs. Cost

As per query we have one API call. The models we are using are OpenAI (`gpt-4o-mini`) and (`gpt-4o`). For embedding model we are using text-embedding-3-small. A simple cost analysis is:

- Routing embedding: ≈$0.004
- SQL generation (`gpt-4o-mini`): ≈$0.001
- Final response (`gpt-4o`, medium length): ≈$0.03–0.08
- Total per query: roughly $0.035–0.085

At 500 queries per week, this costs around $17–42 USD. As it is on prototype level so this is economical but for production grade system we need to look for better optimized options also if we scale to better more accurate models.

The reason of switching between two models is also for cost reduction. Using `gpt-4o-mini` for the intermediate steps (routing, SQL, profile extraction) is useful as we do not face accuracy issues and also makes this part economical. These steps are well-defined enough that the smaller model handles them well. Same is for information parsing from query which can also be handled with this type of small model.

Table 3: Mean latency per stage (50 queries).

Stage	**ms**	**%**
Routing	1,167	18
Retrieval	1,644	25
LLM generation	5,672	87*
Total	**6,483**	

During testing we also switched `gpt-4o` to `gpt-4o-mini` for response generation. This reduced the per-query cost by roughly 80% but produced noticeably shorter responses that missed nuance in the retrieved data. So eventually current split seemed a reasonable middle ground.

3.2 Accuracy vs. Latency

Mean end-to-end latency is around 6.5 seconds per query. Table 3 breaks this down to different stages of pipeline.

LLM generation is the most time taking step. This has different reasons including network bandwidth and we noticed that this time also varies between different parts of the day. The routing and retrieval stages are fast enough that they do not meaningfully affect user experience. The Server-Sent Events streaming approach helps because the user sees the first tokens within roughly 1 to 2 seconds of the LLM starting output. So the 6.5-second total is less disruptive than it would be with a blocking response.

Hybrid queries have slightly higher retrieval latency than single-path queries (both DB and vector search run in sequence). However the optimzing step here is that hybrid is only triggered when routing scores are close so it applies to a minority of queries.

There was on outier case visible in the evaluation. These were few complex DB queries that returned many results and required long responses. As a result it pushed total latency to 13 to 15 seconds. These are edge cases but worth noting for UX design.

3.3 Complexity vs. Generality

The routing mechanism is currently tuned to four intent categories relevant to scholarship discovery. To add another for example of immigration queries internship listings we would require writing a new

anchor vector description and adding a corresponding processing node. Each new route needs enough representative anchor text to separate cleanly from the existing ones.

The PostgreSQL schema is fully domain-specific. Fields like `funding_type`, `scholarship_available` and `degree_level` are meaningful for academic programmes and nothing else. Extending the system to job postings or housing would require a complete schema redesign. The vector search side is more portable but the SQL generation prompts are tied to the current schema.

The current design trades generality for depth. It's core lies in academic programme discovery to be precise than trying to be a general information retrieval system. We think that the alternative is a system that does many things passably so this trade-off is defensible.

4 Failure Cases and open problems

4.1 Where Routing Goes Wrong

Here we achieved 85% accuracy compared to the expected paths it should follow. Some queries like "what are the tuition fees for programmes focused on renewable energy?" contains strong DB signals ("tuition fees") alongside a semantic component ("focused on renewable energy"). The router sends it to the database. The database can filter by the *Renewable Energy* field category but cannot distinguish a programme that is deeply focused on the topic from one that merely lists it as a subject area. The user gets a list of programmes in the category which is close to correct but not quite right. A hybrid route would have been better here.

4.2 SQL Generation Limitations

SQL generation had a 0% error rate on the 50 test queries. Howevery, these were tested in a controlled environment. In real production environment students ask things that do not map cleanly to the schema. For example "find programmes that started recently," "programmes with a flexible application timeline," or "programmes similar to what I studied." These queries either return empty results silently or trigger a misroute to vector search where they may or may not find something useful.

4.3 Needs more data with continous updates

Currently, the data in database was scrapped with many manual interventions. This however is a major bottleneck of system. There are many sources with different site structures. Some sites have all data on single page. Other require navigating multiple pages to find relevant information. So we need to design out scrapping and ETL pipelines to be able to handle new sources as well as continuous updates of old sources.

4.4 Failure on follow up queries

The system currently manages only specific previous queries but cannot handle major follow up conversational queries. This problem considered as Context-aware queries problem requires query reformulation when such type of query is detected and them modified so it can carry context for better retrieval. However system currently cannot handle such queries.

4.5 Open Problems

Vector search returns the top-5 results by cosine similarity to the query. This ranks by relevance but ignores factors the student might also care about university reputation, programme duration, acceptance rate, or scholarship value. A re-ranker which can fuse semantic similarity with structured signals for this particular domain would improve result ordering, especially for vague queries where multiple programmes score similarly in semantic space.

5 Ethical, Efficiency, or Societal Considerations

5.1 Access and Equity

Erasmus Mundus and DAAD programmes are specifically focused to increase access to higher education especially for students from developing countries and underrepresented backgrounds. The purpose of this tool is to lower the search and discovery barrier for these programs. This will have a genuine positive equity angle as it can help students who do not have university counselors or alumni networks that know about these opportunities.

The main weakness of system here is that it is entirely in English. This immediately excludes a large portion of potential users who are not comfortable speakers of english. The data underlying the program are also mostly in English. Multilingual support at minimum for Arabic, Spanish, French and Portuguese will expand reach substantially.

5.2 Field Taxonomy Bias

The 31-category field taxonomy was defined manually based on the programme distribution in the dataset. We noticed that some fields are likely over-represented (Computer Science, Data Science, AI) while others may be missing entirely. As a result a student which some interdisciplinary field may not be able to get precise results. A student in an interdisciplinary field like computational social science or science policy may end up finding their area classified under something adjacent but not quite right.

5.3 Privacy

The system collects a user profile (degree level, field, specialization) during the conversation. This is stored temporarily for chat session for now and nothing is persisted to disk or to any external service between sessions. However for production deployment would need a clear privacy policy which requires explicit consent for profile collection and user decision about whether profile data should persist across sessions.

6 Individual Contributions and Learning Reflection

6.1 Contributions

Responsibilities were divided as follows.

Haris Ali led the data pipeline and structured retrieval layer:

- **Data extraction and ETL**: Developed different scrappers to handle dynamic sources of Eramsus and DAAD catalogue. Also built ETL pipeline to parse and normalize the necessary and relevant data.
- **PostgreSQL data source**: For storage of structured data designed the relational schema and implemented the structured retrieval path. Also wrote the SQL generation prompts that translate natural language filter queries into database queries.

Uzair took the responsibility for semantic search and retrieval experiments.

- **Semantic search**: Designed and implemented the Milvus vector index, including embedding strategy (`text-embedding-3-small`, 1536 dimensions). He also defined chunking parameters (4000-token chunks with 200-token overlap) and inner-product similarity search configuration.
- **Retrieval algorithm experimentation**: Here Uzair performed experiments comparing different retrieval algorithms along with configurations like chunk sizes, overlap strategies, top-k settings, and routing anchor designs to arrive at the final system parameters.

Both of us collaborated on the LLM generation and prompt engineering eventually ensuring quality response generation. We jointly designed the response generation prompts and structured output format. Also added faithfulness constraints to get quality and grounded responses.

6.2 What Worked Well

Firstly using a routing approach to manage different paths for different types of queries helped in making the system efficient in terms of both compute and accuracy. Although the approach of using cosine similarity is simple but with simple tuning it helped us in optimizing retrieval providing useful context with respect to what the user query requires.

Profile enrichment also had a bigger impact on retrieval quality. Enriched queries consistently surfaced more relevant results. It is one of the simpler parts of the implementation but one of the more impactful.

The structured output format for responses also helped in ensuring consistent output format. By adding a restricted design we ask model to only fill specific fields (university name, field, deadline, tuition, URL) rather than write freely. It helped in reducing the number of ungrounded statements and made responses easier to scan.

6.3 What We Would Do Differently

The most significant gap here is the development of more robust and hard baselines and their evaluation. For this we need proper inter annotator agreement scores on our predicted data to identify how good our scores are for real world use. Also we need a bigger sample set of queries collected from actual users and also we should have compared these metrics against systems like chatgpt and other similar domain systems to compare the results. We gave majority of time to development and ignored this important component.

Also we needed to add more components for students. Currently it provides only chat. We should

have added some more notification or smart alerting tools for users to mark the opportunities they are most interested in so the tool is always connected to them and providing them necessary information.

6.4 Learning Reflection

While using a piece of technology we never realize how many different small components need to work in harmony for it to create a useful output. Our first thought was to use llm for everything in it. But later we realized how much uncertainity that adds in out system. Eventually we had to development different small components to make different stages of pipelines deterministic so uncertainitiy can be minimized.

Building a simple RAG versus building a complete chat system with dynamic inputs and multiple data sources was a complete experience as well. We learned alot about LangGraph as a framework and its use in building multi-node agentic systems. Also we explored how different types of data needs different representation based on the problem and semnatic is not the only solution for every RAG.

Eventually it also taught us gaps in our learning and planning. How we lacked in managing and putting effort on systems evaluation and following right protocols. We learned the importance of these aspects and how to also focus on these parts of project.

You can find the complete project here: https://github.com/harisbuilds/ScholarScout.git